

6. Hausaufgabe

Aufgabe 1: Konzepte

Erklären Sie in eigenen Worten, was Sliding-Window, TCP Tahoe, TCP Reno und TCP Vegas sind, wie sie funktionieren, und wofür man diese Techniken braucht. Schreiben Sie ihre Beschreibung in eine Textdatei oder PDF und zeichnen Sie mindestens eine Grafik, die irgendeines dieser Konzepte verdeutlicht. Schreiben Sie außerdem eine Liste aller Protokolle, die in der Vorlesung vorgekommen sind, und in welcher Schicht des ISO/OSI-Modells diese sich befinden (mit kurzer Begründung).

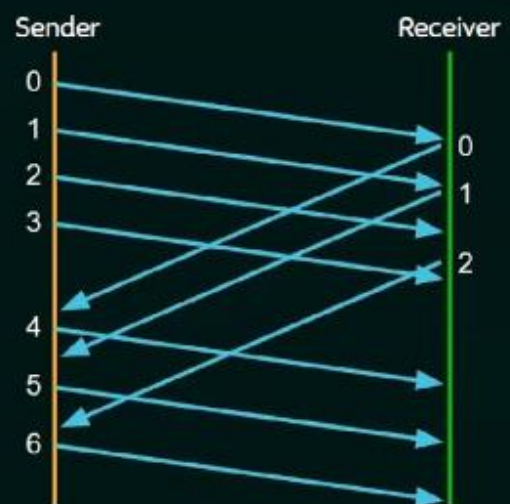
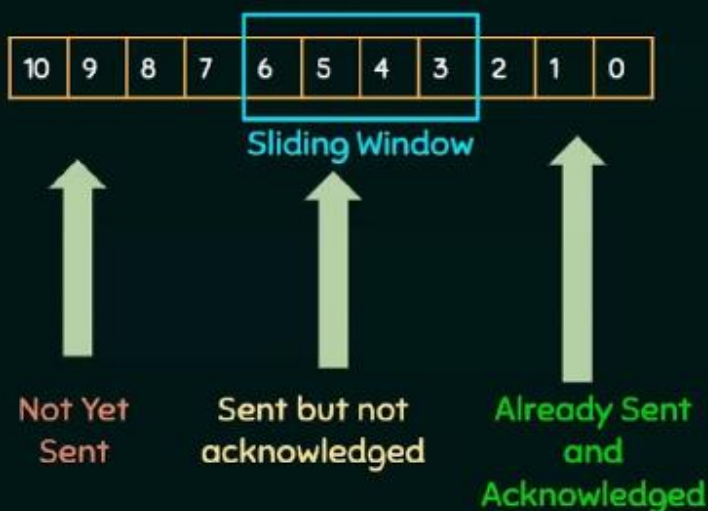
Sliding-Window:

Das Sliding Window ist ein grundlegendes Konzept, das sowohl für die Flusskontrolle als auch für die Staukontrolle in TCP verwendet wird.

Sender und Empfänger haben beide eine Fenstergröße. Das Senderfenster bestimmt, wie viele Daten der Sender gleichzeitig senden darf, während das Empfangsfenster angibt, wie viele Daten der Empfänger aktuell empfangen kann.

Beim erfolgreichen Paketempfangen sendet der Empfänger ein ACK-Paket und das Fenster verschiebt sich um ein Paket.

Sliding Window Protocol



TCP Tahoe:

TCP Tahoe ist ein älteres und grundlegendes TCP-Stauprotokolle. Dabei handelt es sich um einen Algorithmus, welcher eine Überlastung im Netzwerk erkennt und drauf reagiert.

Die Funktionsweise ist in 3 Phasen eingeteilt:

1. Slow Start: Ziel, verfügbare Bandbreite finden (CWND-Wert = Congestion Window)
2. Stauvermeidung: bei überschreitendem CWND-Wert, Bandbreite vorsichtig ausnutzen
3. Schnelle Neuübertragung: Verlust erkennen und Netzwerk entlasten, beginnen bei 1

TCP Tahoe ist effektiv aber aggressiv und wird deshalb nur in Netzen eingesetzt, wo Paketverluste auf Überlastung zurückzuführen sind.

TCP Reno:

TCP Reno ist eine Weiterentwicklung von TCP Tahoe, welche die Leistung bei Paketverlusten verbessern möchte, somit handelt es sich um einen verbesserten Algorithmus, welcher versucht, die Bandbreitennutzung aufrechtzuerhalten. Dabei bleiben Schritt 1 und 2 identisch zu TCP Tahoe, aber im 3. Schritt wird nicht der Traffic runtergefahren und bei Schritt 1 gestartet, sondern man verringert die Anzahl der Pakete nur etwas und hält die Pipeline im Netzwerk gefüllt und den Datendurchsatz bei.

Damit ist TCP Reno besser für Netzwerke geeignet, wo der Paketverlust nicht immer auf eine Überlastung zurückzuführen ist.

TCP Vegas:

TCP Vegas ist ein Staukontrollalgorithmus, der sich grundlegend von Tahoe und Reno unterscheidet, da er versucht, Überlastung **zu vermeiden**, anstatt darauf zu **reagieren**.

Die Funktionsweise sieht wie folgt aus:

1. Messen der kürzesten Bestätigungszeit eines Pakets (RTT)
2. Erwarteter Durchsatz berechnen: $CWND / RTT$
3. Messen des tatsächlichen Durchsatzes
4. Differenz dieser Werte bestimmen, welcher als Maß für wartende Datenpakete gilt
5. CWND-Anpassung (Vergleich mit Schwellenwert Alpha und Beta:
 - CWND erhöhen, wenn Differenz klein ist
 - Differenz zwischen Alpha & Beta, CWND beibehalten
 - CWND erniedrigen, wenn Differenz zu groß

Ziel ist es, CWND so einzustellen, dass die Warteschlange leicht gefüllt sind, aber nicht zu viel. Da Vegas versucht, Staus vorzubeugen und nicht drauf zu reagieren, ist es eine beliebte Variante in Netzwerken. Allerdings kann es auch zu vorsichtig sein und die Bandbreite nicht vollausschöpfen.

Liste mit Protokollen

Ebene	Protokolle
3 Network Layer	IP / ICMP / ARP / RARP Begründung: Host-to-Host Protokoll
4 - 7 Transport Layer – Application Layer	UDP / TCP / SFTP / SSH / SMTP / DNS / NTP / http Begründung: End-to-End Protokoll

Aufgabe 2: Nmap

Finden Sie folgende Dinge mithilfe von Nmap heraus. Falls Sie Windows verwenden, werden Sie eventuell zusätzlich dieses Tool benötigen, das Ihnen Informationen über IP-Adressen liefern kann. Verwenden Sie es mit „whois <ipadresse>“. Beschreiben Sie die einzelnen Nmap-Befehle und deren Befehlselemente (-v, -T4, etc.), die Sie verwendet haben. Sehen Sie sich dazu auch in Wireshark die Pakete an, die dabei gesendet werden.

- a) Wie viele Hosts befinden sich in ihrem lokalen Klasse-C-Netz?
- b) Welches Betriebssystem wird von scanme.nmap.org verwendet?
- c) An welchem Datum wurde die Webseite nmap.org registriert?
- d) Wie kann man möglichst effektiv eine größere Menge an Adressen nach offenen TCP-Ports scannen?
- e) Wie funktioniert der SYN-Scan und für was kann man ihn verwenden?
- f) Welches sind die offenen Ports, die bei Ihren bisherigen Nmap-Scans am häufigsten auftreten, und wofür werden sie verwendet?

- a) **nmap -sN 192.168.178.0/24** → -sN führt einen Ping-Scan durch, bei mir sind es 11 Hosts
- b) **nmap -O scanme.nmap.org** → -O führt OS-Scan durch, Website läuft mit Linux
- c) **whois nmap.org** → Creation Date: 18.01.1999 um 00:00
- d) **nmap -sS --min-rate 1000 -T4 <IP-Adresse oder CIDR>**
Der SYN-Scan (-sS) (Half-Open) ist sehr effektiv, da er mit einer eingestellten Mindestrate und -T4 den Scan aggressiver macht und somit beschleunigt.
- e) Der SYN-Scan ist schnell, da er nur eine verkürzte Variante des Handshake (1./2. Schritt) ausführt um offene Ports zu finden. Da durch die fehlende Rückantwort keine Verbindung hergestellt wird, wird diese Anfrage auch selten geloggt und können schwer verhindert werden.
- f) Die häufigsten Ports sind natürlich 80 und 443 gewesen, jeweils für **http** und **https**, wenn man Webseiten scannt.

Aufgabe 3: DHCP

Fangen Sie sich mit Wireshark mindestens 4 DHCP-Pakete ein. Nutzen Sie ein Capture-Filter, in der PCAP-Datei sollten sich ausschließlich DHCP-Pakete befinden. Überlegen Sie sich, was sie tun müssen, damit diese Pakete überhaupt gesendet/empfangen werden. Untersuchen Sie die Pakete. Machen Sie Screenshots der „Packet Details“ (der Bereich links unten) und beschreiben Sie in eigenen Worten, was Sie anhand der angezeigten Informationen über das Protokoll sagen können. Schreiben sie ihre Entdeckungen in eine Textdatei oder PDF.

DHCP-Pakete werden grundsätzlich gesendet, wenn ein Client sich mit einem Netzwerk verbindet und noch keine IP-Adresse hat oder eine DHCPDISCOVER-Anfrage gestellt hat.

```
> Frame 1: 342 bytes on wire (2736 bits), 342 bytes captured (2736 bits) on interface \Device\NPF_{C58BA841-F640-47F
▼ Ethernet II, Src: 2a:a3:f6:f3:54:e1 (2a:a3:f6:f3:54:e1), Dst: Broadcast (ff:ff:ff:ff:ff:ff)
  > Destination: Broadcast (ff:ff:ff:ff:ff:ff)
  > Source: 2a:a3:f6:f3:54:e1 (2a:a3:f6:f3:54:e1)
    Type: IPv4 (0x0800)
    [Stream index: 0]
  > Internet Protocol Version 4, Src: 0.0.0.0, Dst: 255.255.255.255
  > User Datagram Protocol, Src Port: 68, Dst Port: 67
▼ Dynamic Host Configuration Protocol (Request)
  Message type: Boot Request (1)
  Hardware type: Ethernet (0x01)
  Hardware address length: 6
  Hops: 0
  Transaction ID: 0x12f5c700
  Seconds elapsed: 0
  > Bootp flags: 0x0000 (Unicast)
  Client IP address: 0.0.0.0
  Your (client) IP address: 0.0.0.0
  Next server IP address: 0.0.0.0
  Relay agent IP address: 0.0.0.0
  Client MAC address: 2a:a3:f6:f3:54:e1 (2a:a3:f6:f3:54:e1)
  Client hardware address padding: 00000000000000000000
  Server host name not given
  Boot file name not given
  Magic cookie: DHCP
  > Option: (53) DHCP Message Type (Request)
  > Option: (55) Parameter Request List
  > Option: (57) Maximum DHCP Message Size
  > Option: (61) Client identifier
  > Option: (50) Requested IP Address (192.168.178.148)
  > Option: (51) IP Address Lease Time
  > Option: (12) Host Name
  > Option: (255) End
  Padding: 0000000000000000000000000000
```

Aufgefallen ist, dass bei allen DHCP-Paketen UDP als Host-to-Host Protokoll verwendet wird und meistens über die Ports 67 und 68 gesendet werden.

Der Client identifiziert sich mit seiner MAC-Adresse und fordert mit **Requested IP Address** eine neue IP-Adresse an.

Diese Beobachtungen treffen auch auf die anderen Pakete zu.

Aufgabe 4: Routing

Informieren Sie sich über dynamische, und insbesondere distanzvektorbasierte Routingverfahren. Die Kosten zur Datenvermittlung zu anderen Rechnern werden in einem (Distanz-)Vektor in einer Tabelle gespeichert, die man Routingtabelle nennt. In den Graphen werden Router als Knoten, Kosten als Kanten dargestellt.

Erklären Sie mit Hilfe der Routingtabellen und dem Graphen des Netzes, was in jedem Schritt passiert, indem Sie einen Router raussuchen und die Veränderungen pro Runde analysieren und mit den Tabellen der anderen Router vergleichen. Entwickeln Sie darauf aufbauend einen Algorithmus (in Worten), der die Veränderungen in den Routingtabellen vollzieht. Was sind die Unterschiede zu Link-State-Verfahren?

Bei der Initialisierung wird für jeden Knoten eine Tabelle mit Kosten (pro Hop) angelegt und alle Wege, welche nicht existieren, ausgegraut. Im Anschluss werden die Pfade erweitert, zum Beispiel kommt man von A zu C via B, wobei hier immer der kürzeste Weg markiert wird. Dies wird solange wiederholt, bis alle möglichen Felder ausgefüllt sind.

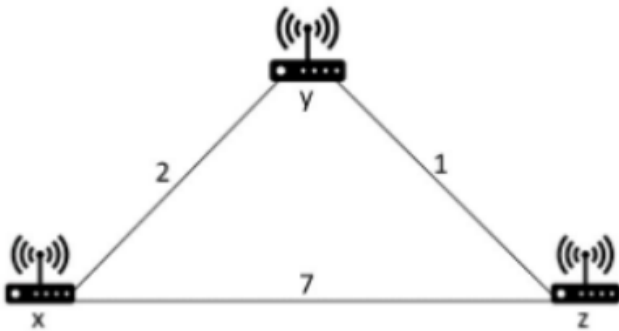
Algorithmus (Pseudo-Code):

- 1) For each Knoten n in Graph G: erstelle eine Tabelle
 - pro Knoten n erstelle Zeile „zu n“
 - pro Knoten n erstelle Spalte „via n“
- 2) ergänze Distanzen der direkten Nachbarn eintragen
- 3) Aktualisiere solange die Tabelle, indem du alle möglichen Pfade abgehst
 - Addition der Kantengewichte
 - pro Zeile kleinsten Wert markieren
- 4) Fertig, sobald alle Pfade abgegangen sind
 - markierten Pfade in Zeile zeigen günstigsten Weg an

Bei **Link-State-Verfahren** erhält jeder Router eine globale Karte des gesamten Netzwerks und kennt somit jeden Router, jede Verbindung und alle Kosten. Zudem teilen die Router den eigenen Status der Verbindung mit allen anderen Routern.

Bei **distanzvektorbasierten Verfahren** kennt jeder Router nur seinen direkten Nachbarn und die Entfernung zu ihm. Somit ist hier keine globale Übersicht von Routern enthalten. Zudem teilen die Router den eigenen Status der Verbindung nur mit den direkten Nachbarn.

- a) Bestimmen Sie die Routingtabellen in jedem Schritt, indem Sie Ihren zuvor entwickelten Algorithmus anwenden. Geben Sie danach den kostengünstigsten Weg von Router „z“ zu Router „x“ an.



Von x	Via x	Via y	Via z
Zu x	 	 	
Zu y	 	2	
Zu z	 	 	7

Von y	Via x	Via y	Via z
Zu x	2	 	
Zu y	 	 	
Zu z	 	 	1

Von z	Via x	Via y	Via z
Zu x	7	 	
Zu y	 	1	
Zu z	 	 	

Von x	Via x	Via y	Via z
Zu x	 	 	
Zu y	 	2	8
Zu z	 	3	7

Von y	Via x	Via y	Via z
Zu x	2	 	8
Zu y	 	 	
Zu z	9	 	1

Von z	Via x	Via y	Via z
Zu x	7	3	
Zu y	9	1	
Zu z	 	 	

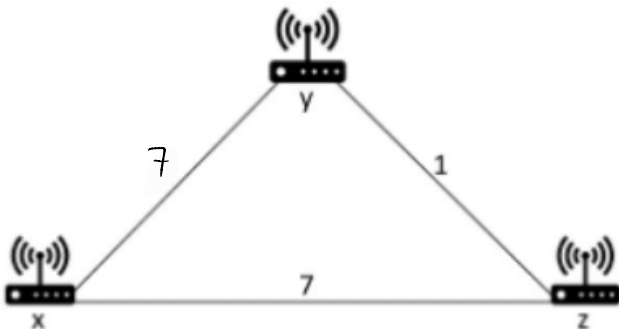
Von x	Via x	Via y	Via z
Zu x	 	 	
Zu y	 	2	8
Zu z	 	3	7

Von y	Via x	Via y	Via z
Zu x	2	 	8
Zu y	 	 	
Zu z	9	 	1

Von z	Via x	Via y	Via z
Zu x	7	3	
Zu y	9	1	
Zu z	 	 	

günstigster Weg von z zu x über y: Kosten 3

- b) Die Kosten zwischen „x“ und „y“ steigen nun von 2 auf 7. Berechnen Sie die Routingtabellen mit Hilfe des Algorithmus. Ändert sich der kostengünstigste Pfad von „z“ nach „x“?



Von x	Via x	Via y	Via z
Zu x	2	2	2
Zu y	2	7	
Zu z	2		7

Von y	Via x	Via y	Via z
Zu x	7	2	
Zu y	2	2	2
Zu z		2	1

Von z	Via x	Via y	Via z
Zu x	7		2
Zu y		1	2
Zu z	2	2	2

Von x	Via x	Via y	Via z
Zu x	2	2	2
Zu y	2	7	8
Zu z	2	8	7

Von y	Via x	Via y	Via z
Zu x	7	2	8
Zu y	2	2	2
Zu z	14	2	1

Von z	Via x	Via y	Via z
Zu x	7	8	2
Zu y	14	1	2
Zu z	2	2	2

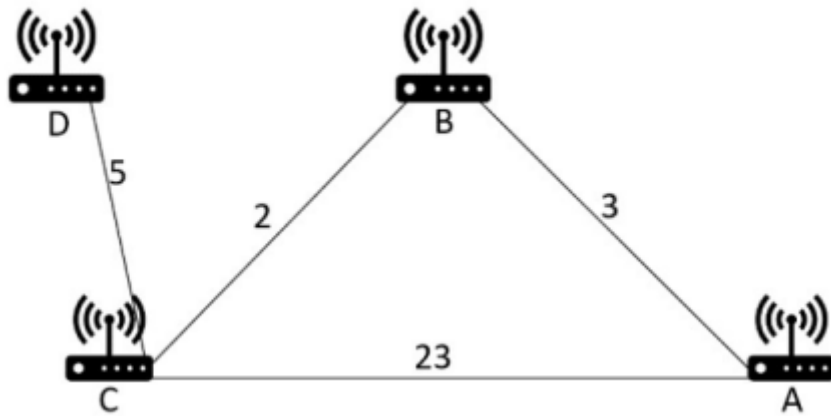
Von x	Via x	Via y	Via z
Zu x	2	2	2
Zu y	2	7	8
Zu z	2	8	7

Von y	Via x	Via y	Via z
Zu x	7	2	8
Zu y	2	2	2
Zu z	14	2	1

Von z	Via x	Via y	Via z
Zu x	7	8	2
Zu y	14	1	2
Zu z	2	2	2

Der günstigste Pfad von z nach x ändert sich und läuft jetzt direkt von z zu x mit Kosten 7.

- c) Sehen Sie sich den unteren Graphen an. Router „D“ fällt nun auf einmal aus. Beschreiben Sie, ob und wann die anderen Router merken, dass keine Verbindung mehr zu „D“ möglich ist.



C fällt es als direkter Nachbar sofort auf.
Die Router A & B würden es erst in der 2 Iteration bemerken, sofern sie D schon davor gekannt haben.